

E-Commerce Sales Insights System

SQL-first relational analytics + interactive local stakeholder dashboard

PORTFOLIO STACK	DATA PERIOD	DEPLOYMENT
SQLite • SQL • Python Standard Library	January - June 2026	Local SQLite database; no Oracle/MySQL/PostgreSQL server required

Project Summary

This project models a complete e-commerce analytics environment covering customers, products, orders, order items, payments, marketing channels and spend, website sessions, shipping, returns, and inventory. SQL views and analytical queries transform transaction-level data into decision-ready KPIs, while a lightweight local dashboard presents those results for business users.

Why SQLite? The same relational design can be implemented in PostgreSQL or MySQL, but this packaged portfolio version uses SQLite so the database ships as a single file and runs without a database server. Primary/foreign keys, constraints, joins, views, indexes, aggregations and CTE-based analytics are still demonstrated.

Executive KPI Snapshot

Paid Orders	Net Revenue	AOV	Units Sold	Gross Margin	Repeat Rate
838	\$53,343.76	\$63.66	1,786	64.58%	39.34%

1. Architecture & Design

Layer	Implementation
Data Layer	SQLite relational database (database/ecsales.db)
Modeling Layer	Normalized tables with PK/FK/CHECK constraints
Semantic Layer	Reusable views for paid order lines, order summaries, channels and customer LTV
Analytics Layer	15 SQL analyses for sales, marketing, cohorts, refunds, shipping, payments and inventory
Presentation Layer	Local dashboard: Overview, Marketing, Products, Customers, Operations and Interactive SQL Lab

Core entities

Entity	Purpose	Rows
customers	Customer profile, geography, signup and acquisition channel	700
products	SKU, category, cost and list price	31
orders	Order header, status, channel and destination	949
order_items	Line-level quantity, selling price and discount	1,576
payments	Payment amount, method and status	949
shipping	Courier, promise, delivery date/cost/status	949
returns	Return reason and refund value	47
website_sessions	Sessions, carts, checkouts and conversions by day/channel	1,086
channel_spend	Daily channel spend, impressions and clicks	1,086

The model separates order headers from order lines, keeps product cost in the product master, and stores transaction selling price and discount at line level. Marketing spend and website sessions are daily facts by channel, which enables conversion, ROAS and CAC analysis.

2. KPI & Business Logic

Metric	Definition	Business use
Net Revenue	Paid line revenue: (unit price × quantity) - discount	Tracks realized paid sales
AOV	Net revenue / paid order count	Measures typical basket value
Gross Profit	Line revenue - product cost	Measures product economics
ROAS	Attributed revenue / channel spend	Compares marketing efficiency
CAC	Channel spend / new customers	Evaluates acquisition cost
Conversion Rate	Conversions / website sessions	Measures funnel efficiency
Repeat Rate	Customers with 2+ paid orders / purchasing customers	Signals retention
30-day Cohort Rate	Customers reordering within 30 days / cohort size	Measures early repeat behavior
Late Delivery Rate	Late delivered shipments / delivered shipments	Measures fulfillment service level

Example analytical SQL

```
SELECT channel_name,  
ROUND(SUM(revenue),2) AS revenue,  
ROUND(SUM(spend),2) AS spend,  
ROUND(SUM(revenue)/NULLIF(SUM(spend),0),2) AS roas  
FROM v_channel_daily  
GROUP BY channel_name  
ORDER BY roas DESC;
```

3. Marketing Performance

Channel	Revenue	Spend	ROAS	Conversion %
Organic Search	\$7,055.14	\$6,821.77	1.03	3.72%
Email	\$6,816.51	\$8,617.44	0.79	4.88%
Meta Ads	\$13,429.51	\$67,509.78	0.20	2.91%
Google Ads	\$14,085.75	\$81,806.24	0.17	3.36%
Influencer	\$2,883.45	\$34,374.14	0.08	2.62%
Direct	\$9,073.40	\$0.00	-	4.13%

Channels should not be ranked on revenue alone. Revenue, spend, ROAS, CAC and conversion answer different questions about scale, efficiency and funnel quality. The dashboard preserves undefined ROAS when media spend is zero rather than inventing a value.

4. Product, Customer & Operations Insights

Top products by paid revenue

Product	Category	Units	Revenue
Mechanical Keyboard	Electronics	63	\$5,291.12
Smart Watch	Electronics	60	\$4,537.84
Denim Jacket	Fashion	60	\$2,964.32
Portable Speaker	Electronics	46	\$2,783.52
Power Bank 20000mAh	Electronics	68	\$2,522.72

Customer analytics

The database contains 516 purchasing customers, of whom 203 made at least two paid purchases. The repeat-customer rate is 39.34%. The customer LTV view exposes paid order count, lifetime revenue, average order value, first order date and last order date for ranking high-value customers.

Shipping performance

Courier	Delivered	Avg days	Late %
DTDC	210	4.73	43.81%
Ecom Express	189	4.75	43.92%
BlueDart	204	4.70	44.12%
Delhivery	235	4.73	46.81%

5. Interactive Dashboard

Page	Decision support
Overview	Revenue, AOV, margin, product and regional performance
Marketing	Channel revenue, spend, ROAS, CAC, sessions and conversion
Products	Category/product profitability and inventory risk
Customers	Lifetime value, repeat purchase and 30-day cohort retention
Operations	Courier service levels, returns/refunds and payment mix
SQL Lab	Ad hoc read-only SQL execution against the bundled database

6. Interactive SQL Lab - Live Query Execution

The upgraded application includes an interactive SQL workspace rather than only displaying fixed SQL examples. A user can choose a prepared query from the Query Library or type/modify a custom **SELECT** or **WITH** statement in the editor. When **Run Query** is clicked, the application executes the statement against the bundled SQLite database and renders the returned rows immediately.

What the SQL Lab returns: the result table, execution time in milliseconds, and number of rows displayed. Results are calculated from the current local project database at execution time, so the output changes when the query logic or filters change.

Example custom query

```
SELECT p.product_name, ROUND(SUM(oi.quantity * oi.unit_price - oi.discount), 2) AS revenue
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
JOIN orders o ON o.order_id = oi.order_id
WHERE o.status = 'PAID'
GROUP BY p.product_id, p.product_name
ORDER BY revenue DESC
LIMIT 10;
```

Safety design

The public SQL workspace is intentionally **read-only**. It accepts analytical SELECT/WITH statements but blocks database-changing operations such as INSERT, UPDATE, DELETE, DROP and ALTER. This lets reviewers explore the data without accidentally damaging the packaged database. The application also limits displayed output to 200 rows for a responsive demonstration.

Query Library

Prepared examples make the project easy to demonstrate quickly. A reviewer can load an existing business query, inspect or modify it, and execute the changed SQL. This combines guided examples with genuine ad hoc analysis.

Meaning of 'real-time' in this project

Real-time means the SQL is executed when the user clicks Run Query and the result is calculated immediately from the local SQLite database. It does **not** claim that live orders are being streamed from an external e-commerce company or internet API.

7. SQL Engineering & Optimization

The project uses indexes on frequent join/filter keys, including order status/date, customer ID, channel ID, order-item order/product IDs, product category, marketing spend date/channel, shipping order and return product. Views reduce duplicated KPI logic and create a stable semantic layer for the dashboard.

Key SQL techniques demonstrated

- Primary keys, foreign keys and CHECK constraints
- INNER JOIN and LEFT JOIN across transactional/reference tables
- SUM, COUNT, AVG and conditional aggregation
- CTEs for cohort and multi-stage calculations
- Subqueries and reusable views
- Date grouping and date-difference logic
- NULL-safe division using NULLIF
- Indexes for join/filter performance
- Read-only ad hoc query execution through the interactive SQL Lab

Portability note

SQLite uses `date()`, `strftime()` and `julianday()` in this package. A PostgreSQL migration would replace these with `date casts`, `DATE_TRUNC` and `interval/date arithmetic`. The relational design and KPI equations remain conceptually the same.

8. Running the Project

Extract the project ZIP and double-click **run_dashboard.bat** on Windows. The application starts a local Python HTTP service and reads the bundled SQLite database. No third-party Python package and no separate SQL server are required. Keep the terminal window open while using the dashboard.

9. Future Enhancements

Potential production upgrades include PostgreSQL or a cloud warehouse, automated ETL, scheduled refresh, role-based access, data-quality checks, multi-touch attribution, customer segmentation, forecasting, recommendation features, and deployment behind authenticated web access.

Appendix A - Public Portfolio Deliverables

File / Folder	Purpose
database/ecsales.db	Ready-to-use relational SQLite database
sql/01_schema.sql	DDL for core entities and constraints
sql/02_views_indexes.sql	Reusable semantic views and optimization indexes
sql/03_analytics_queries.sql	15 business analytics queries
data/*.csv	Source-table exports for inspection or BI import
app.py	Interactive local dashboard application
run_dashboard.bat	Windows one-click launcher
docs/Project_Report.pdf	Public project report
README.md	Repository overview, architecture, setup and usage

Portfolio privacy decision

Private interview-preparation questions and answers are intentionally **not included** in this public package or project report. They can be maintained separately for personal preparation.

Dataset disclaimer

All data in this portfolio package is synthetic and generated for demonstration. Example names, emails, orders, marketing activity, shipping events and return records do not represent real customers or companies.